Introduction to Computer Science

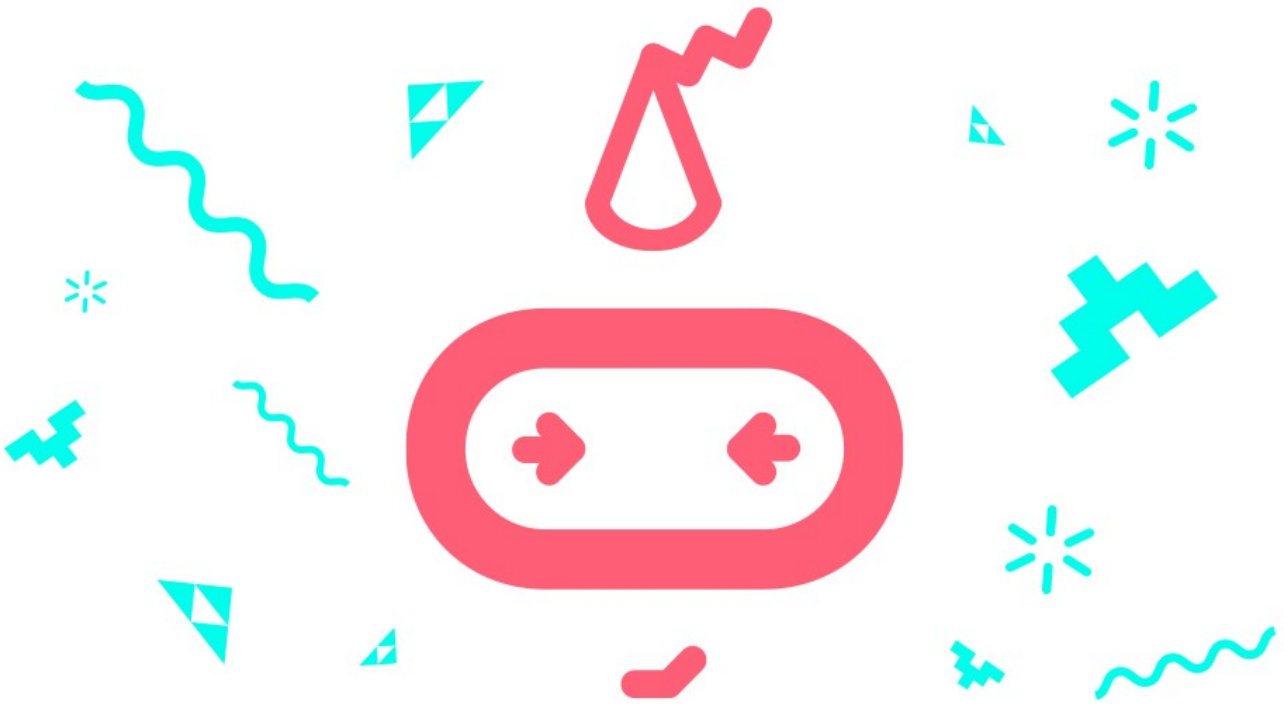
Unit 10:

Radio

Student workbook

makecode.microbit.org

Table of Contents



Overview

Unit summary

This unit introduces the radio functionality of the micro:bit and covers the use of more than one micro:bit to share and combine data. The unplugged activity is a version of the traditional Hotter/Colder call-and-response game. In the coding activities, you will send and receive number and string data through the Radio blocks. Up to this point, you have been challenged to collaborate independently while you create your own projects. This unit's project is a great opportunity for you to work with a partner to design a program to send and receive some sort of data to and from each other through your micro:bits. There is a wide range of simple and complex projects you can try, but whatever you choose, it is a whole lot of fun to communicate with other people using micro:bits!

Learning goals

During this unit, you will:

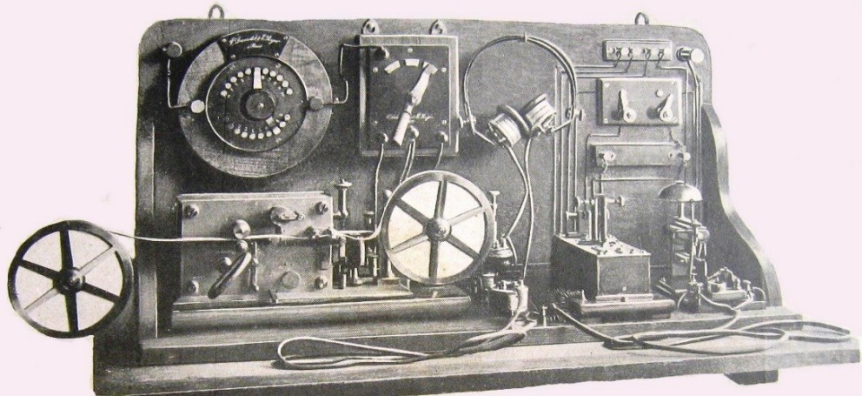
- Understand how to use the Radio blocks to send and receive data between micro:bits.
- Understand the specific types of data that can be sent over the radio.
- Work in pairs to apply the above knowledge and skills to design a unique program using radio communication between two micro:bits.

Use this space to write your questions and ideas

Lesson A: Understanding radio communication

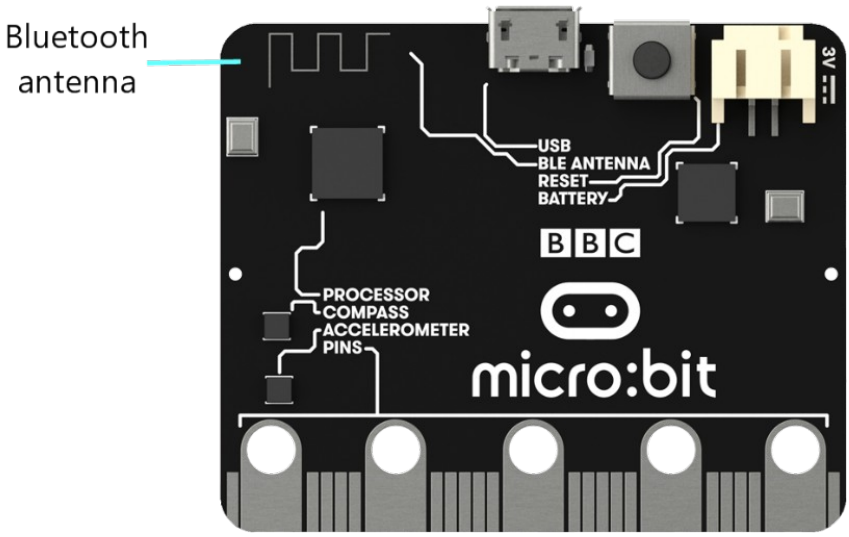
Overview: Radio communication

Radio communication uses electromagnetic waves to send messages through the air. A radio communication system requires a transmitter to send messages and a receiver to receive messages. The first “radios” were invented in 1894 and used radio waves to transmit and receive telegraph signals. This is an image of one of the first radios:



The micro:bit allows you to communicate with other micro:bits in the area using the blocks in the Radio category. The micro:bit can act as a sender and/or a receiver, depending on what blocks you use in your program. You can send a number, a string (a word or series of characters), or a string/number combination in a message. You can also give a micro:bit instructions on what to do when it receives a radio message.

On the back of the micro:bit, if you look closely in the upper left corner, there is an antenna that enables the micro:bit to communicate.



Lesson B: Explore the radio toolbox

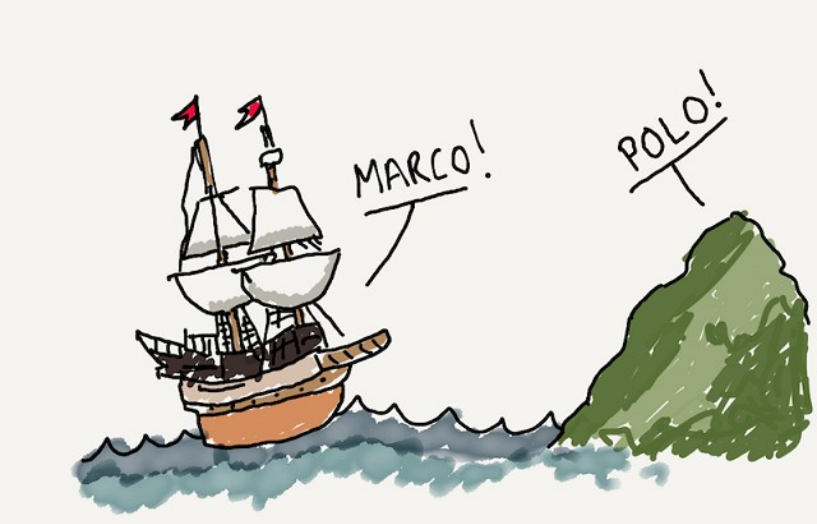
Overview of the coding activities

The radio communication blocks enable you to send and receive data between two micro:bits. You'll do two activities today to code sending and receiving different types of data:

- Marco Polo – Send and receive string data between micro:bits
- Morse code – Send and receive number data between micro:bits

Coding activity 1: Marco Polo

Marco Polo was the first Westerner to journey to eastern Asia and document his travels. There is an American game called "Marco Polo", which is a form of call-and-response tag played in a swimming pool. One person closes their eyes and calls "Marco," and the other players must respond "Polo." Using the sound of their voices only, the Marco player must find and tag the Polo players.



You will be playing a form of this "Marco Polo" game using the radio on the micro:bits.

This activity focuses on using Radio blocks to send and receive strings between micro:bits.

Set group ID number

When communicating between micro:bits, it is important that the micro:bits involved are all using the same group ID—think of this as the radio frequency or channel across which the micro:bits will communicate. So, the first thing you will do is set the group ID number.

1. In Microsoft MakeCode, start a new project and name it: Marco Polo. Either delete the ‘forever’ block in the coding Workspace or move it to the side, as it’s not used in the activity.
2. Then from the Radio Toolbox, drag a ‘radio set group’ block to the coding Workspace and connect into the ‘on start’ block. In the ‘radio set group’ block, leave the default value of 1 for the group ID.

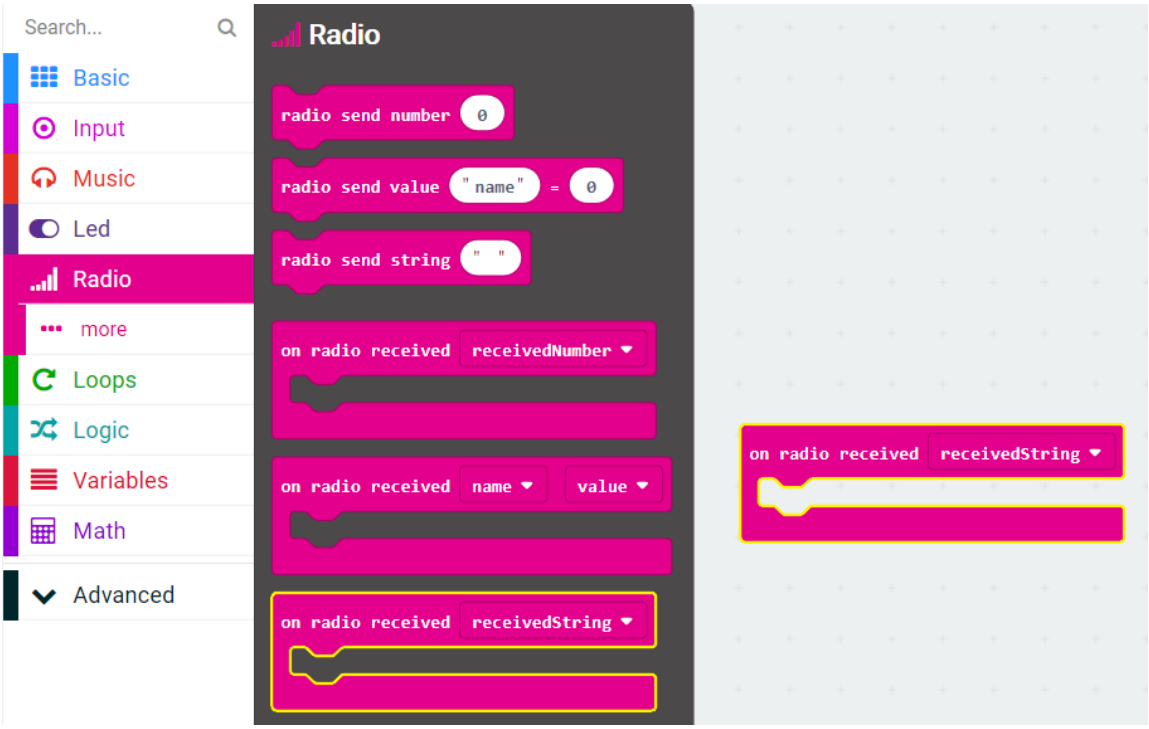
Code radio send for button A and button B

3. Drag two ‘on button pressed’ blocks to the coding Workspace. Leave one with the default value A , and use the dropdown menu to change the other button to B.
4. From the Radio Toolbox drawer, drag two ‘radio send string’ blocks to the coding Workspace. Place one ‘radio send string’ block into the ‘on button A pressed’ block, and the other ’radio send string’ block into the ‘on button B pressed’ block. Then:
    a. In the ‘on button A pressed’ block, change the default empty string value of the ‘radio send string’ block by typing the string: Marco
    b. In the ‘on button B pressed’ block, change the default empty string value of the ‘radio send string’ block by typing the string: Polo

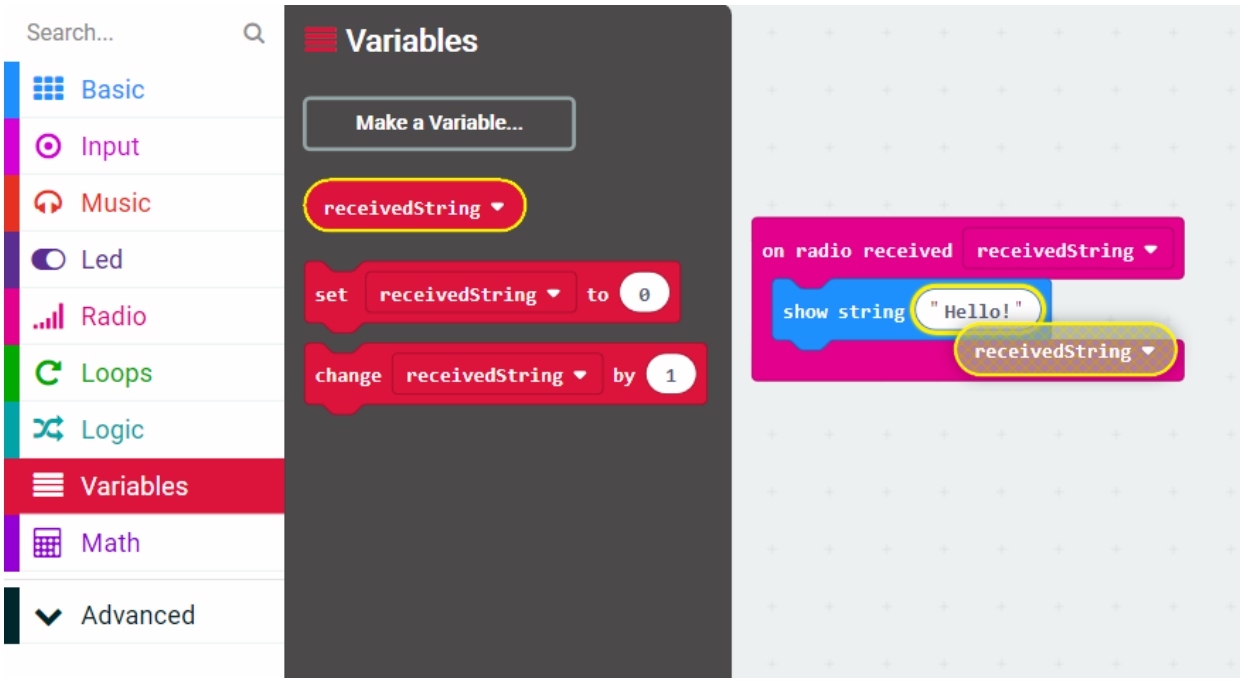
Code radio received

5. To display the data sent between the micro:bits, drag an ‘on radio received (receivedString)’ block to the coding Workspace.

Note: There are a lot of blocks in the Radio category that look similar. Make sure you use the ‘on radio received’ block with ‘received*String*’ value.



6. From the Basic Toolbox drawer, drag a ‘show string’ block into the ‘on radio received (receivedString)’ block. Then, from the Variables Toolbox drawer, drag a ‘receivedString’ variable block to replace the default string value of “Hello” in the ‘show string’ block.



Test the code

Run the code in the Simulator first. Then, download to the program to the micro:bits and test.

> When using the Radio blocks, the micro:bit Simulator will show two micro:bits
>
> In the Simulator, a radio transmission icon will appear in the top-right corner of the micro:bit. The icon will light up as the micro:bit is transmitting data.
>
> In the Simulator, all the code in the coding Workspace runs on both virtual micro:bits. You should include code for how to send data as well as what to do when it receives data.

Play

In this version of the game, there will be multiple Marcos and only one Polo.

- You and your fellow students should connect your micro:bits to a battery pack so you can walk around the room.
- Your teacher will secretly assign one student to be “Polo.”
- All students should walk around the room and press button A.
- The single “Polo” student should be the only one to press button B. You need to find out who the Polo student is!

Mod

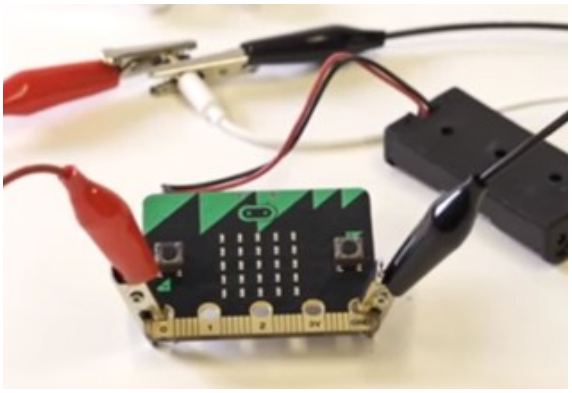
This mod will play one note when button A is pressed to send the Marco string to the second micro:bit, and a different note when button B is pressed to send the Polo string.

a. Add a ‘show leds’ block to the ‘on start’ block to show an image of the initials MP.

b. From the Music Toolbox drawer, drag 2 ‘play tone’ blocks to the coding workspace.

c. Drag one of the ‘play tone’ blocks to the ‘on button A pressed’ block, and the other one to the ‘on button B pressed’ block.

d. Change the default value in the ‘play tone’ block that is inside the ‘on button A pressed’ block to the value Low C.

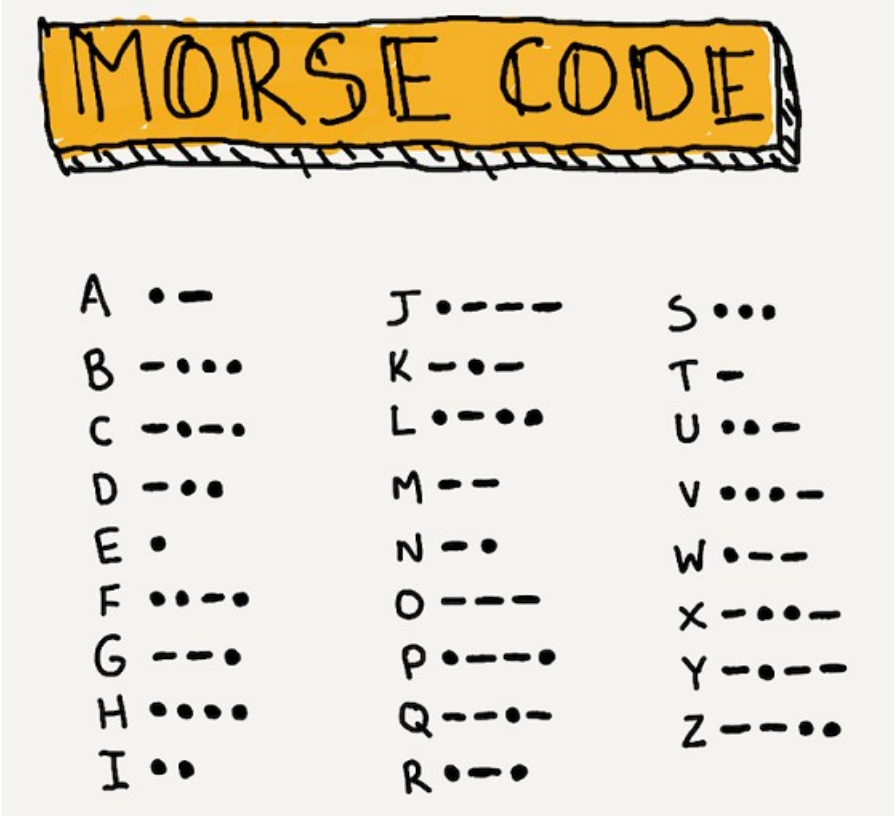


When you run the code in the Simulator, notice how it shows the headphone attached to the micro:bit pins!

Note: To run the program on the micro:bit with sound, you need to connect it to some speakers or headphones. See how to hack your headphone at makecode.microbit.org/projects/hack-your-headphones and follow the instructions.

Coding activity 2: Morse code

Morse code is a character encoding scheme used in telecommunication that encodes text characters as standardized sequences of two different signal durations called dots and dashes. Morse code is named for Samuel F. B. Morse, an inventor of the telegraph. The first versions were invented in the early 1800s and has been refined since then. In the late 1800s, it was used for early radio communication before it was possible to transmit voice.

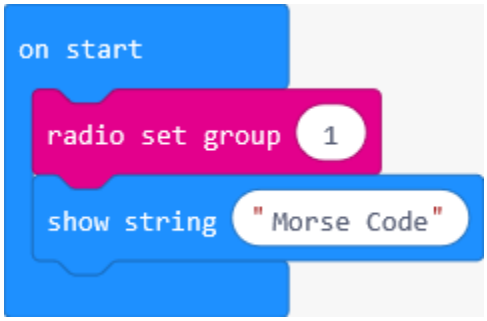


This activity focuses on using Radio blocks to send and receive numbers between micro:bits:

- Depending on the button pressed, a different number value is sent between micro:bits.
- On receiving a number, the micro:bit will display a different image unique to the number sent.
- One number will represent a dot, another a dash, and another a space or stop.

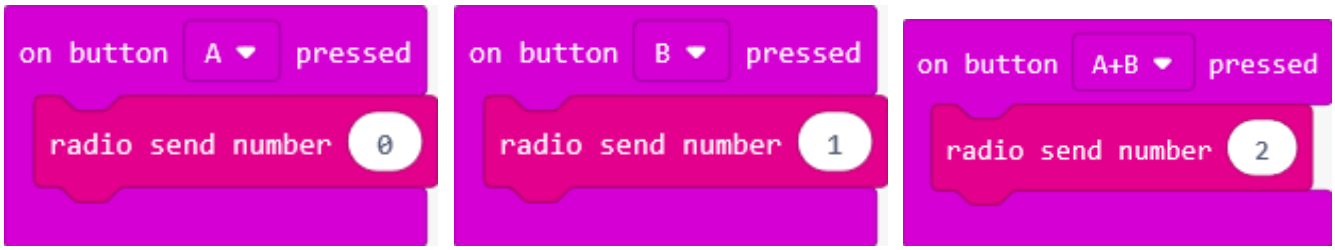
Set the group ID

1. In Microsoft MakeCode, start a new project and name it: Morse code. Either delete the ‘forever’ block in the coding Workspace or move it to the side, as it’s not used in the activity.
2. Set the Radio group ID number, following the same steps as the previous activity. Then add a ‘show string’ block to the ‘on start’ block, to identify the program. In this example, the default string value of Hello is changed to the value Morse Code.



Code the buttons

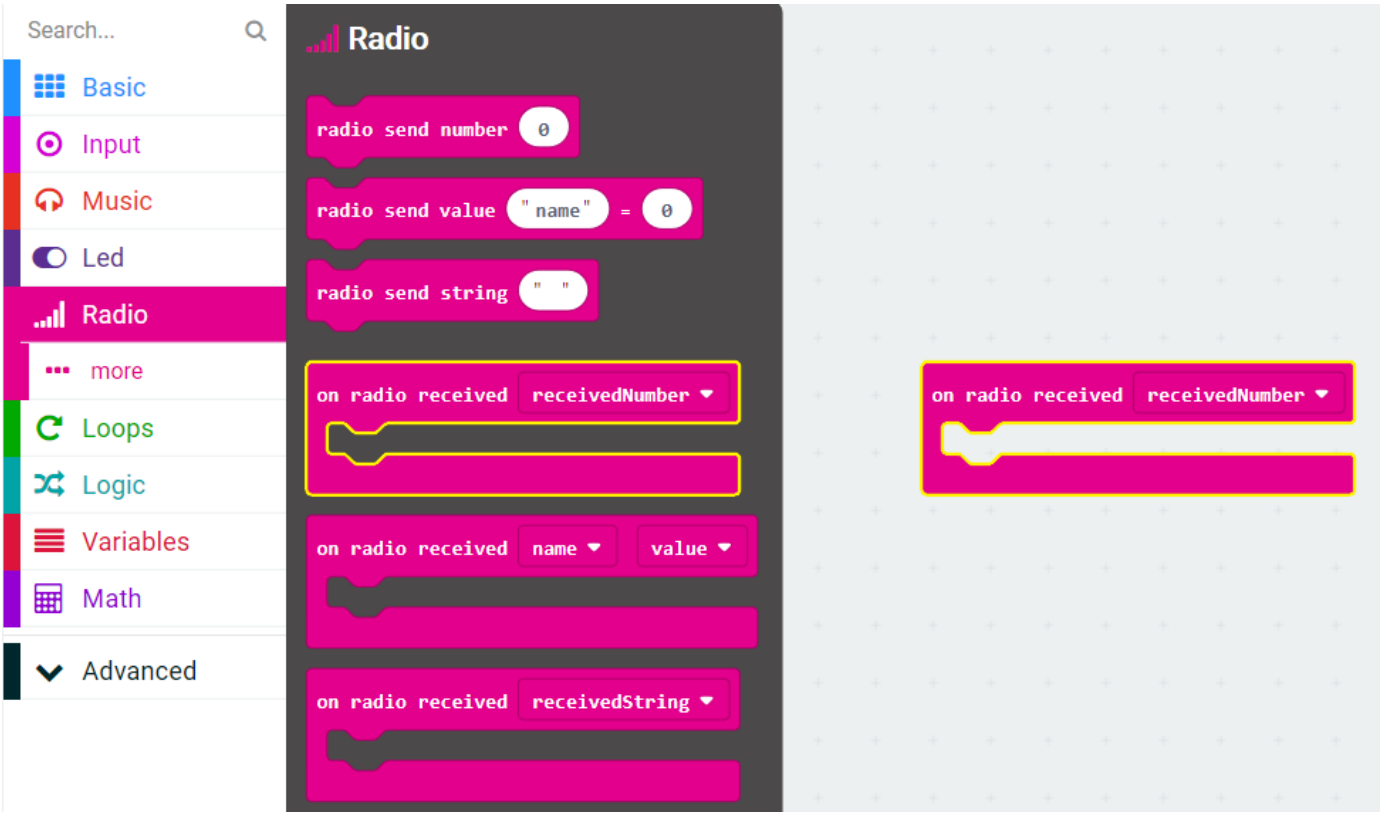
3. Drag three ‘on button pressed’ blocks to the coding workspace. Leave one with the default value A, change the value in the second block to B, and change the value in the third block to A+B.
4. From the Radio Toolbox drawer, drag three ‘radio send number’ blocks to the coding workspace and place one radio send number block into each of the ‘on button pressed’ blocks.
    a. In the ‘on button A pressed’ block, leave the default number value of the ‘radio send number’ block as 0.
    b. In the ‘on button B pressed’ block, change the default number value of the ‘radio send number’ block to the value 1.
    c. In the ‘on button A+B pressed’ block, change the default number value of the ‘radio send number’ block to the value 2.



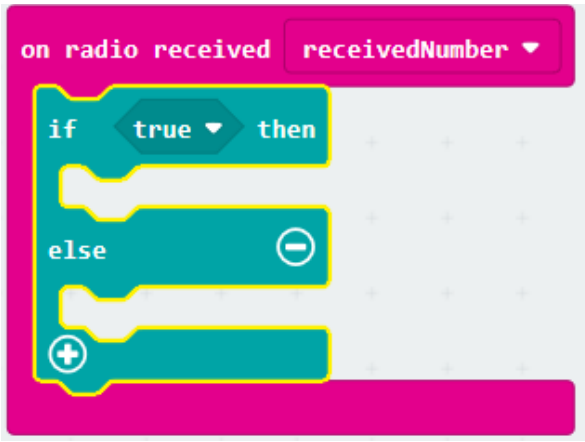
Code radio received

5. From the Radio Toolbox drawer, drag an ‘on radio received (receivedNumber)’ event handler to the coding Workspace.

Note: There are a lot of blocks in the Radio category that look similar. Make sure you use the block with the ‘received*Number*’ value.

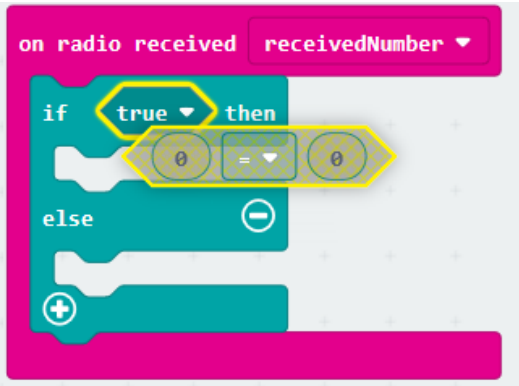


6. Since you will display a different image depending on the number value received, you need a logic block. From the Logic Toolbox drawer, drag an ‘if…then…else’ block to the coding Workspace and place it in the ‘on radio received (receivedNumber)’ event handler.

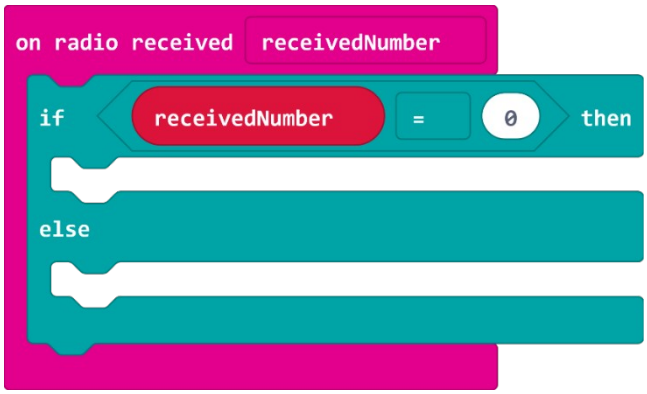


In order to know whether to display a dot, a dash, or a space/stop image, you need to compare the number received to the values 0, 1, and 2.

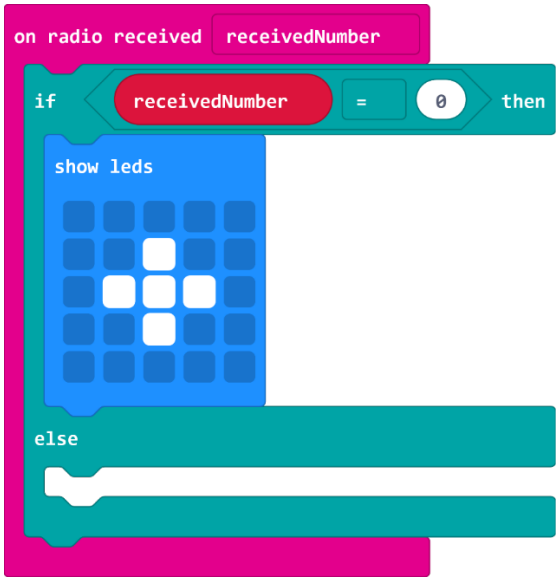
7. From the Logic Toolbox drawer, drag a ‘0=0’ comparison hexagon block onto the coding Workspace and drop it into the ‘if…then’ block replacing the default value of ‘true’.



8. From the Variables Toolbox drawer, drag a ‘receivedNumber’ variable block onto the coding Workspace, and drop it into the first slot of the equals comparison block. Leave the default value of 0 in the second slot.



9. From the Basic Toolbox, drag a ‘show leds’ block to the coding Workspace and drop it under the ‘if…then’ clause. Then, create an image to represent a dot.

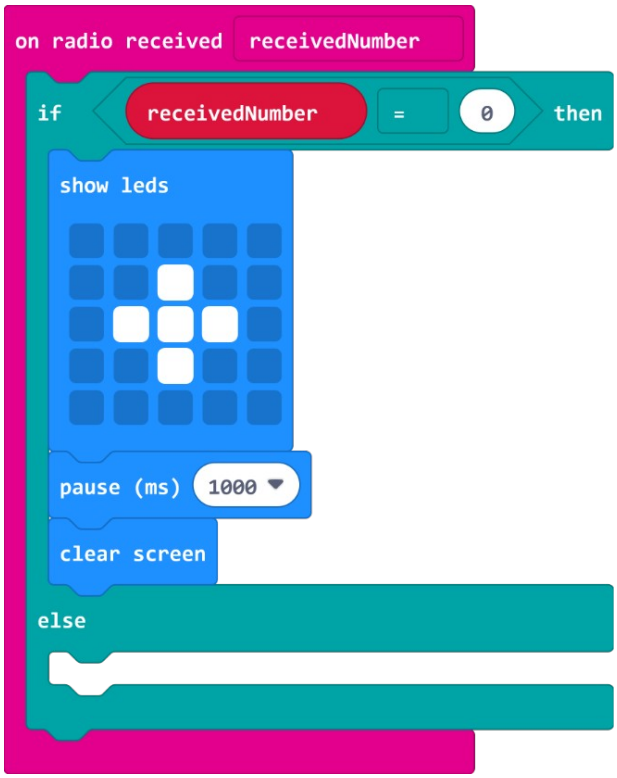


Try it!

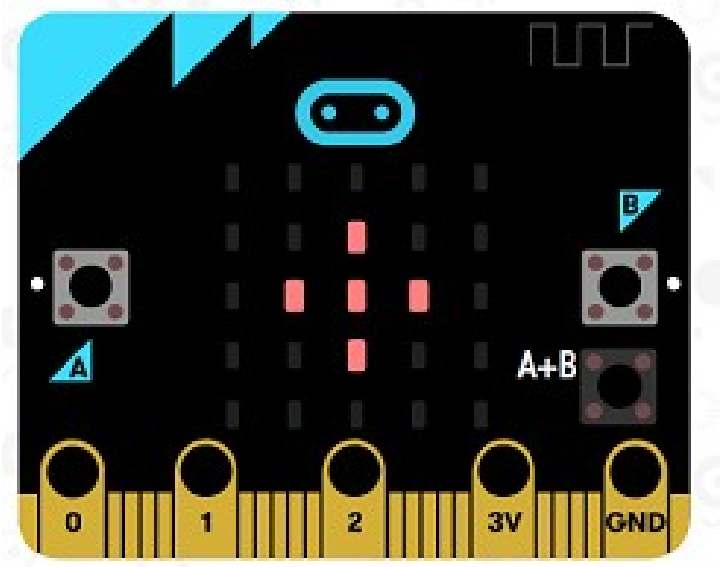
Download the program to the micro:bit and press button A on the sending micro:bit. Does this cause a dot to be displayed on the receiving micro:bit?

However, pressing button A again does not appear to send another dot, as the image on the receiving micro:bit does not appear to change.

Challenge question: How can you fix this?



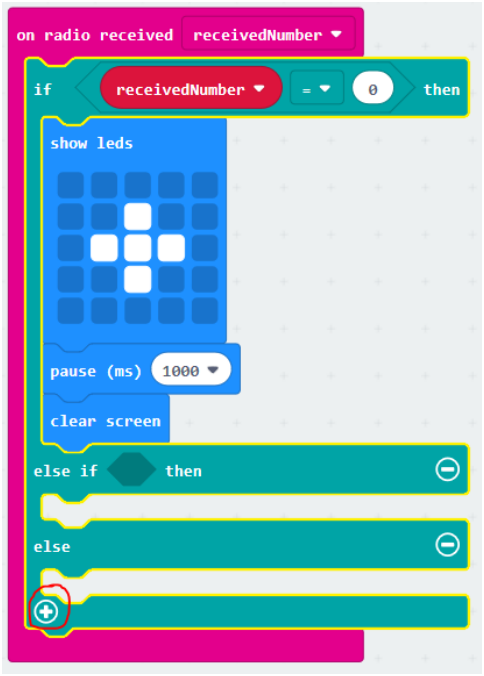
Try running the program again. Now each time the sender presses button A, you see a dot appear.



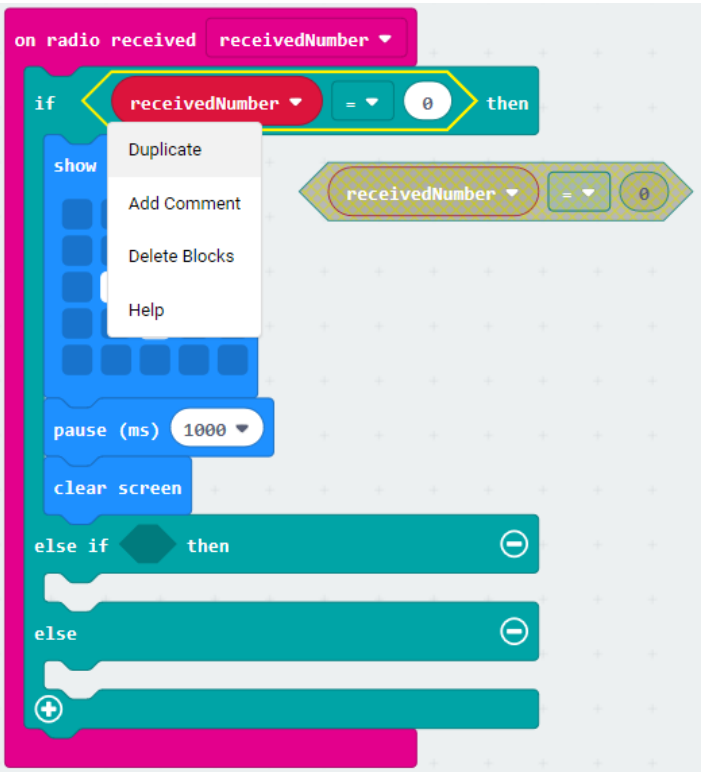
Code the other received images

Now you need to specify what to display if you receive a 1 or 2.

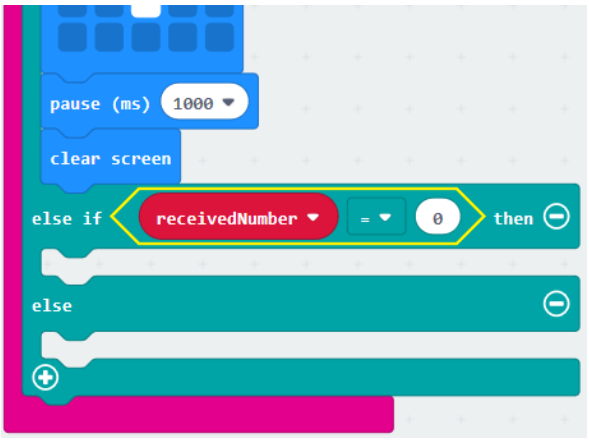
10. In the ‘if…then…else’ block, select the plus (+) icon to create an ‘else if’ clause.



11. Right-click on the equals comparison block in the ‘if’ clause and select Duplicate to create a copy.



12. Then, drag this new equals comparison block into the ‘else if’ clause.



13. Change the values in the second slot of the equals comparison block from 0 to 1.

Note that you don’t have to test ‘receivedNumber’ value in the third ‘else’ clause—this is because if the value is not 0 or 1, then it must be 2, since there are only three possibilities.

14. From the Basic Toolbox drawer, drag ‘show leds’, ‘pause’, and ‘clear screen’ blocks to the ‘else if’ and ‘else’ clauses. Then, modify the ‘show leds’ images displayed:
    a. For the ‘else if (receivedNumber=1)’, show a dash.
    b. For the ‘else’ clause (which is when the ‘receivedNumber’ variable equals 2), show a full screen of lights.

Try it!

Download your program to the micro:bit. Press buttons A, B, and A+B together on the sending micro:bit to see the associated image on the receiving micro:bit.

Mod

A final else

In a conditional that might receive a number of different values, it is good coding practice to have a catch-all ‘else’ clause. In the example, if any number value other than the ones you coded for (0,1, and 2) is received, you can signal the user that an error has occurred by using a ‘show icon’ block to display an X.

The pause and clear screen

15. Rather than repeat these lines of code three times, you can move the ‘pause’ block and the ‘clear screen’ block outside of the edited ‘if…then…else’ block and inside the ‘on radio received (receivedNumber)’ block.

Now your program runs as you designed it to run and is more efficient, too! Download the revised program to the micro:bits and test it out.

Lesson C: Make a micro:bit radio

Overview: Pair programming

Computer programmers will often collaborate on a project together. It's called "pair programming." Here's how it works:

- Two people share one computer with each person having a different role.
- The person in the navigator role provides instructions.
- The person in the driver role is at the keyboard typing the code from the navigator.
- It's important that the navigator does not take control of the keyboard unless the driver agrees.
- Good communication and partnering is critical.
- Be sure to switch roles so each person has the opportunity to play both roles. For larger projects, the pair may switch roles many times during development and testing.

Activity: micro:bit radio

For this project, you will work in pairs to design a project that incorporates radio communication to send and receive data in some way.

- Some projects may have two separate programs: One that receives data, and one that sends data. You and your coding partner might each choose to submit one program in that case.
- In other cases, you and your coding partner might submit one program that has both sending and receiving code in it, and the same code is uploaded to two or more micro:bits.

Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses the radio to send and receive data with meaningful actions and responses for each
- Uses Radio blocks in a way that is integral to the program
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry

The design thinking process:

1. Empathize by learning more about your target audience.
2. Define: Understand and identify your audience's problems or needs.
3. Ideate: Brainstorm several possible creative solutions.
4. Prototype: Construct rough drafts or sketches of your ideas.
5. Test your prototype solutions.

Use this space to start your design thinking process:

Project ideas

1. Stop, thief!

Design an alarm system for your bedroom that alerts you with a screen animation when someone opens your door. You can mount one micro:bit on your door and use the accelerometer to send a signal over the radio when it is being moved.

2. Interactive art

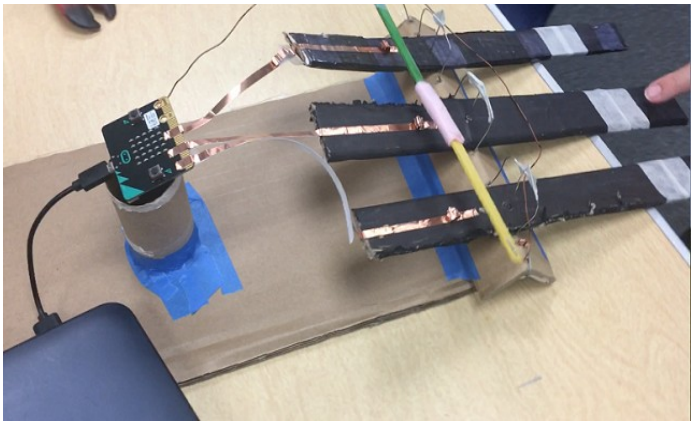
Create a piece of interactive artwork that receives something as input over the radio from another micro:bit and displays something based on that as output.

Project examples

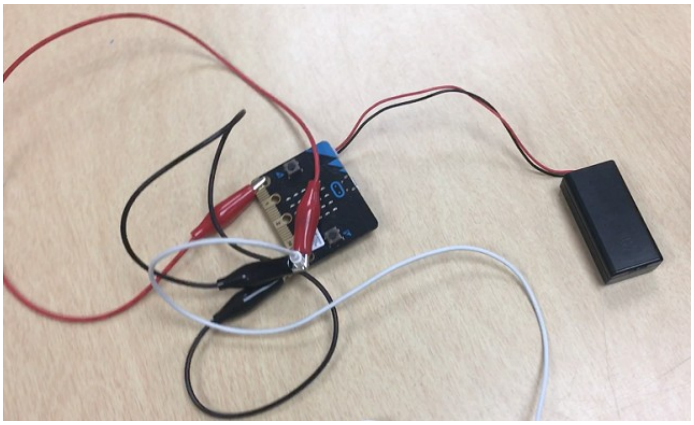
Three-note keyboard

This is a simple three-note keyboard that uses wooden paint stirrers and copper tape to make a connection to each of the three pins on the micro:bit. When a key is pressed, it sends a number over the radio to a second micro:bit that plays the appropriate tone over a set of earbuds. This allows you to use each of the three pins on the first micro:bit to play a different tone.

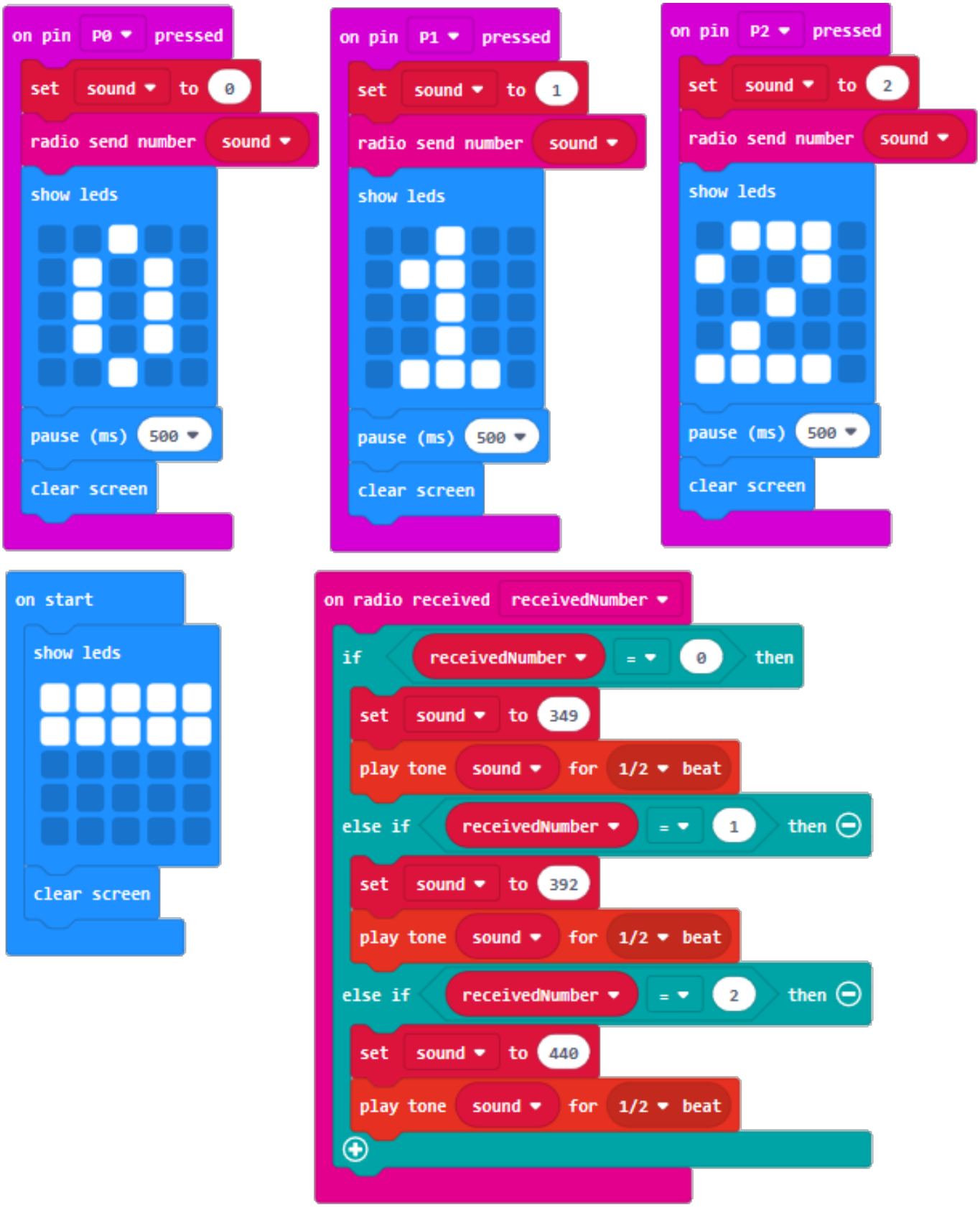
First micro:bit with keyboard and copper tape connections



Second micro:bit that plays the notes on earbuds



Complete program



Solution link: makecode.microbit.org/_iXKbWu8f2H60

Radio tennis

In this project, the tennis racquets alternate displaying a ball on the micro:bit LED screen. When you swing the racquet, the ball disappears from one micro:bit display and shows up on the other micro:bit's display.

Project scoring rubric

Assessment elements	1	2	3	4
Radio	No working and/or meaningful use of radio.	Use of radio is incomplete or non-functional and/or tangential to operation of program.	Effectively uses the radio to send or receive data, with meaningful actions and responses for each.	Effectively uses the radio to send and receive data, with meaningful actions and responses for each.
micro:bit program	micro:bit program lacks three or more of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ▪ Effectively uses the Radio to send and receive data, with meaningful actions and responses for each ▪ Compiles and runs as intended ▪ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- What kind of project did you do? How did you decide what to pick?
- How does your project use radio communication?
- Are there separate programs for the Sender and the Receiver micro:bits? Or one program for both?
- Describe something in your project that you are proud of.
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your testers give you? How did that help you improve your design?
- How would you improve your project, given more time?
- Relating to the pair programming process:
  - What challenges did you encounter working with a partner?
  - What benefits did you gain?
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements, including: ▪ Brainstorming ideas ▪ Construction ▪ Programming ▪ Beta testing

Glossary of key terms

Radio communication: The use of electromagnetic waves to send messages through the air. A radio communication system requires a transmitter to send messages and a receiver to receive messages.